

Approval Components

Reusable approval workflow components built with React, Chakra UI, and TanStack Table.

This README explains how to use the components in a clean, production-friendly way.

🌟 Overview

`ApprovalPage` is a reusable container that provides:

- Data table with selection
- Row click → Drawer view
- Single approve/reject actions
- Bulk approve/reject actions
- Filtering support

You supply the data, configuration, and handlers.

🚀 Quick Start

```
<ApprovalPage
  title="Reassignment Approvals"
  data={data}
  columns={columns}
  rowIdKey="id"
  onApprove={handleApprove}
  onReject={handleReject}
  onBulkApprove={handleBulkApprove}
  onBulkReject={handleBulkReject}
  renderDrawerContent={(row) => <DrawerContent row={row} />}
  filters={filters}
/>
```

📦 Requirements

1. Data Source

Store your table data in state.

```
const [data, setData] = React.useState(MOCK_DATA);
```

Each row must have a unique identifier.

```
rowIdKey="id"
```

Column Configuration

Define columns using `ColumnConfig<T>[]`.

```
const columns: ColumnConfig<Request>[] = [
  { key: "requester", header: "Requester" },
  { key: "type", header: "Type" },
  { key: "department", header: "Department" },
];
```

Custom Rendering

```
{
  key: "status",
  header: "Status",
  render: (v) => <StatusBadge status={String(v)} />,
}
```

Fixed Width Columns

```
{
  key: "id",
  header: "ID",
  width: "120px",
}
```

Filters

Define filters using `FilterConfig[]`.

```
const filters: FilterConfig[] = [
  {
    key: "status",
    label: "All Statuses",
    options: [
      { label: "Pending", value: "Pending" },
      { label: "Approved", value: "Approved" },
      { label: "Rejected", value: "Rejected" },
    ],
  },
];
```

```
    },  
  ];
```

Each filter key must match a field in your data.

Drawer (Row Details)

Control drawer content via:

```
renderDrawerContent={ (row) => <DrawerContent row={row} /> }
```

Example

```
function DrawerContent({ row }: { row: Request }) {  
  return (  
    <VStack align="stretch" gap="4">  
      <DrawerField label="Requester" value={row.requester} />  
      <DrawerField label="Department" value={row.department} />  
      <DrawerField label="Type" value={row.type} />  
    </VStack>  
  );  
}
```

Reusable Field Component

```
function DrawerField({ label, value }: { label: string; value:  
React.ReactNode }) {  
  return (  
    <Box>  
      <Text fontSize="xs">{label}</Text>  
      <Text fontSize="sm">{value}</Text>  
    </Box>  
  );  
}
```

Single Actions

Approve

```
const handleApprove = async (row: Request) => {  
  setData((prev) =>  
    prev.map((r) => (r.id === row.id ? { ...r, status: "Approved" } : r)),
```

```
    );  
  };  
};
```

Reject

```
const handleReject = async (row: Request) => {  
  setData((prev) =>  
    prev.map((r) => (r.id === row.id ? { ...r, status: "Rejected" } : r)),  
  );  
};
```



Bulk Actions

Bulk Approve

```
const handleBulkApprove = async (rows: Request[]) => {  
  const ids = new Set(rows.map((r) => r.id));  
  setData((prev) =>  
    prev.map((r) => (ids.has(r.id) ? { ...r, status: "Approved" } : r)),  
  );  
};
```

Bulk Reject

```
const handleBulkReject = async (rows: Request[]) => {  
  const ids = new Set(rows.map((r) => r.id));  
  setData((prev) =>  
    prev.map((r) => (ids.has(r.id) ? { ...r, status: "Rejected" } : r)),  
  );  
};
```



Optional Feedback

```
toast.success(`${row.id} approved`);  
toast.success(`${rows.length} requests approved`);
```



Example Data Shape

```
interface Request {  
  id: string;
```

```
requester: string;
type: string;
amount: number;
status: string;
date: string;
department: string;
description: string;
}
```

Recommended Flow

1. Define your data type
2. Store data in state
3. Configure columns
4. Configure filters
5. Create drawer content
6. Add approve/reject handlers
7. Plug everything into `ApprovalPage`

Suggested Structure

```
/components/approval/
  ApprovalPage.tsx
  ApprovalDrawer.tsx
  StatusBadge.tsx
  types.ts

/pages/
  approval/index.tsx
```

Best Practices

- Always use a stable `rowIdKey` (e.g., `id`)
- Keep drawer UI separate from table config
- Use `render` for formatting (badges, currency, etc.)
- Separate bulk and single handlers
- Keep components reusable and stateless where possible

Reference

👉 Refer to the `ApprovalPage` component for the full orchestration logic.

Summary

`ApprovalPage` is a flexible approval engine.

You provide: - Data - Columns - Filters - Drawer content - Action handlers

Use it for: - Approval workflows - Reassignment flows - Document approvals - Any table-based decision system

Maintained for internal reuse and consistency across approval modules.